

Sistemas Operativos

Departamento de Computación - FCEyN - UBA
Segundo cuatrimestre de 2025

Nombre y apellido:

Nº orden: — — L.U.

Cant. hojas: 3

Primer parcial - 2do cuatrimestre de 2025

1	2	3	Nota
B-	B	B-	(A)

- Cada ejercicio debe realizarse en hojas separadas y numeradas. Debe identificarse cada hoja con nombre, apellido, LU y número de orden. Hojas sin identificar no serán corregidas. Completar en la hoja de enunciado la cantidad total de hojas entregadas y entregarla junto al examen, la misma no se incluye en la cantidad total de hojas entregadas.
- La devolución de los exámenes corregidos es personal. Los pedidos de revisión se realizarán por escrito, sin retirar el examen corregido del aula.
- Cada ejercicio se califica con Bien, Regular o Mal. La división de los ejercicios en incisos es meramente orientativa y cada ejercicio se califica globalmente. El parcial se aprueba con un mínimo de 2 ejercicios Bien y 1 Regular.

Ejercicio 1.

Considerar el siguiente fragmento de código:

```
int ticket = 0;
int turno = 0;

void proc() {
    cosas_no_criticas();
    int miTurno = obtenerTurno();
    while (miTurno != turno);
    funcion_critica(); // sección crítica
    terminarTurno();
    cosas_no_criticas();
}
```

Dadas las siguientes implementaciones de las funciones `obtenerTurno` y `terminarTurno`:

```
obtenerTurno() {
    int temp = ticket;
    ticket++;
    return temp;
}

terminarTurno() {
    turno++;
}
```

Responder:

- Si dos procesos ejecutan este código de manera concurrente, determinar si al utilizar las funciones `obtenerTurno` y `terminarTurno` se delimita correctamente una sección crítica. En caso contrario, mostrar una traza que ejemplifique el problema. Justificar.
- Si hay un problema con la exclusión mutua, ¿cómo se podría arreglar? Justificar cómo y con qué mecanismos de software/hardware.

Ejercicio 2.

Sea un sistema donde un proceso *parent* crea un *child* y el *child* crea un *grandchild*. El *grandchild* debe imprimir "Soy el mejor nieto", luego dormir por 60 segundos, una vez pasado el minuto avisar al *child* que se despertó usando *pipes* y por último colgarse con un `while(TRUE)`; Al llegar al punto de cuelgue y recibir el mensaje por parte del *grandchild*, el *child* debe matarlo y hacer un *exit*. Por último el *parent* espera a que el *child* termine e imprime "Mi hijo querido se ha ido" y termina.

- Escribir un programa (en pseudocódigo) que se comporte como el sistema descrito, utilizando las syscalls necesarias. Dado que se trata de pseudocódigo, no es necesario ser estricto con los parámetros y los valores de retorno de los syscalls, aunque sí respetar su espíritu.

- b) Describir en qué estado queda cada uno de los procesos involucrados luego de la ejecución de cada syscall.

Ejercicio 3.

Se tiene la siguiente salida para la ejecución de `strace -f` sobre un binario determinado:

```
pipe([3, 4]) = 0
pipe([5, 6]) = 0
clone(Process 4673 attached...) = 4673
[pid 4672] close(3) = 0
• [pid 4673] close(4) = 0
[pid 4672] close(6) = 0
• [pid 4673] close(5) = 0
• [pid 4673] read(3, <unfinished ...>
[pid 4672] read(0, "anda\n", 1024) = 5
[pid 4672] write(4, "anda\n", 5) = 5
• [pid 4673] <... read resumed> "anda\n", 256) = 5
[pid 4672] read(5, <unfinished ...>
• [pid 4673] write(6, "MENSAJE ERRONEO\n", 16) = 16
[pid 4672] <... read resumed> "MENSAJE ERRONEO\n", 256) = 16
• [pid 4673] read(3, <unfinished ...>
[pid 4672] read(0, "vamos\n", 1024) = 6
[pid 4672] write(4, "vamos\n", 6) = 6
• [pid 4673] <... read resumed> "vamos\n", 256) = 6
[pid 4672] read(5, <unfinished ...>
• [pid 4673] write(6, "MENSAJE ERRONEO\n", 16) = 16
[pid 4672] <... read resumed> "MENSAJE ERRONEO\n", 256) = 16
• [pid 4673] read(3, <unfinished ...>
[pid 4672] read(0, "IR!\n", 1024) = 4
[pid 4672] write(4, "IR!\n", 4) = 4
• [pid 4673] <... read resumed> "IR!\n", 256) = 4
[pid 4672] read(5, <unfinished ...>
• [pid 4673] write(6, "YENDO\n", 6) = 6
[pid 4672] <... read resumed> "YENDO\n", 256) = 6
• [pid 4673] rt_sigprocmask(SIG_BLOCK, [CHLD], <unfinished ...>
[pid 4672] wait4(4673, Process 4672 suspended
<unfinished ...>
• [pid 4673] <... rt_sigprocmask resumed> [], 8) = 0
• [pid 4673] rt_sigaction(SIGCHLD, NULL, {SIG_DFL, [], 0}, 8) = 0
• [pid 4673] rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
• [pid 4673] nanosleep({5, 0}, 0x7fff34b442a0) = 0
• [pid 4673] exit_group(0) = ?
Process 4672 resumed
• Process 4673 detached
<... wait4 resumed> [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 4673
--- SIGCHLD (Child exited) @ 0 (0) ---
write(1, "LLEGAMOS\n", 9) = 9
exit_group(0) = ?
```

- Indicar qué *file descriptors* son utilizados, y si vienen definidos por sistema (stdin, stdout, stderr) o son solicitados por el programa.
- ¿Es este programa interactivo? Justificar.
- Escribir un programa que al seguirlo con `strace` produzca la misma salida.

(Ej. 1) (a) No se delimita correctamente la sección crítica. Dependiendo de como el scheduler otorgue tiempo de ejecución a los dos procesos podría pasar que ambos ejecuten la sección crítica al mismo tiempo.

Troza:

Memoria compartida
inicial

ticket = 0
turno = 0.

P₁

P₂

int temp = ticket; // temp = 0

int temp = ticket; // temp = 0

ticket++

| ticket = 1 |

return temp.

miTurno = 0

0 = 0 ✓ (miTurno ≠ turno)

<Sección crítica>

| ticket = 2 | ticket++

return temp

miTurno = 0

0 = 0 ✓

<Sección crítica> (miTurno ≠ turno).

(b) Solución conservando el mecanismo de turnos.

En el punto (a) vemos que obtenerTurno en sí mismo es una sección crítica. Asimismo, terminarTurno también podría tener algunos problemas (race condition) como lo que estamos evaluando (aunque para ese caso no estoy mostrando ninguna troza).

Podemos: 1) Proteger obtenerTurno con un mutex y terminarTurno con el mismo mutex: m₁ y ~~m₂~~: ~~mutex~~ En memoria compartida (el mismo para los dos procesos). (mutex = Sem(1)). Este es el mec. x Software →

2. utilizar enteros atómicos: no permiten que un proceso de load, inc, store sea "partido" por el scheduler. ticket y turno serian enteros atómicos y se podrian usar alguna de sus primitivos para obtener una copia incrementada del valor en el caso de ticket. (no estoy seguro si existe) y simplemente ++ en el caso de turno. (Este es el mecanismo x hardware).

Implemente el primer

✓ ++ retorna el valor previo e incrementa el valor de modo atomico (en enteros atomicos).

m = Sem(1).

void proc () {

no_critico()

m.wait()

miTurno = obtenerTurno();

m.signal();

while (miTurno != turno);

funcionCritica();

m.wait()

terminarTurno();

m.signal();

✓
→ Si obtenerTurno funciona correctamente no es necesario proteger terminarTurno ya que no sera ejecutado concurrentemente.

3.

Ej 2 a

(b)

(b) Estados después de `syscall`:

```
int main() {
```

```
if (fork() == 0) {
```

```
// child
```

```
int fds[2];
```

```
pipe(fds);
```

```
pid_t id = fork();
```

```
if (id == 0) {
```

```
// grand child
```

```
close(fds[READ]);
```

```
Print("Soy el mejor nieto");
```

```
sleep(60);
```

```
write(fds[WRITE], 1, sizeof(int)); // creo que podría no haber
```

```
close(fds[WRITE]);
```

```
while(1);
```

```
// no sería necesario un exit
```

```
porque nunca debería llegar.
```

```
// child
```

```
close(fds[WRITE]);
```

```
int r
```

```
read(fds[READ], &r, sizeof(int));
```

```
close(fds[READ]);
```

```
Kill(id, SIGTERM);
```

```
exit(0);
```

```
}
```

```
// parent
```

```
wait(NULL);
```

```
Print("Mi hijo querido se ha ido");
```

```
exit(0);
```

fork: Se genera un nuevo proceso hijo que se ejecuta concurrentemente al padre.

pipe: Se generan 2 file descriptors para el proceso que lo corre. Forma parte de su estructura.

close: Se eliminan las referencias creadas a pipe propios del proceso.

sleep: El scheduler no otorga tiempo de computo por una cont. aproximada a la especificada al proceso q' lo corre.

write: Se escribe en el fd una cont. de bytes.

// creo que podría no haber
// usado esto y directamente
// cerrar el fd pues read
// es bloqueante.

• AGREGO
(b)

READ: Se leen bytes x un fd.

kill: Se envia una señal al proceso con pid = id.

wait: El proceso padre queda en espera hasta que 1 hijo muera (SIGCHLD).

Exit: El proceso en cuestión es desdorado con código error. El hijo puede vivir si no es controlado.

AGREGO antes de entregar (punto b)

Algunos syscalls cambian el "estado" de un proceso.

Para hacer más clara mi respuesta:

- SLEEP y WAIT.

El proceso pasa de "running" a "paused" hasta que se de una condición y el scheduler le cambie el estado nuevamente.

- Fork, deja el proceso hijo creado en algún estado (no recuerdo el nombre) que lo representa antes de empezar a correr.

- exit: el proceso pasa a "finished" o un estado que representa que finalizó.

Ej 3 a) File descriptors utilizados: 0, 1, 3, 4, 5, 6.

(Stdin y Stdout (0 y 1 respectivamente) en uso.

Stderr no está en uso (2).

El resto son solicitados por el programa (3, 4, 5, 6).

b) Dado que hay multiples lecturas de la terminal por Stdin(0) y una escritura a la terminal, Stdout(1): Considero que es interactivo, pues para que funcione alguien tiene que

escribir.

idea principal del programa:

hay 2 procesos: padre e hijo. Ambos se comunican con 2 pipes de la siguiente forma: fd1: descriptors para que el padre hable con el hijo; fd2: descriptors para que el hijo hable con el padre. El padre toma un input de la terminal, se lo envia al hijo por el pipe y el hijo determina si es correcto o no. Se repite hasta que se acierta.

c)

```
int main() {
    int fd1[2] // 3, 4
    int fd2[2] // 5, 6
```

```
    pipe(fd1)
```

```
    pipe(fd2)
```

```
    if (fork() == 0) {
```

```
        // hijo.
```

```
        close(fd1[WRITE])
```

```
        close(fd2[READ])
```

```
        char b1[256]
```

```
        while (read(fd1[READ], b1, sizeof(b1)) > 0)
```

```
        while (read(fd1[READ], b1, sizeof(b1))) {
```

```
            if (b1 != "IR!\n") {
```

Str Error

```
                write(fd2[WRITE], "MENSAJE ERRONEO\n", 16);
```

```
            } else {
```

```
                write(fd2[WRITE], "XEND\n", 6);
```

```
                break;
```

```
            }
```

```
        }
```

// el hijo hace algo más que no entiende el dar con el mensaje.
// sigprocmask (SIG_BLOCK ...) SIG_BLOCK: no lo tengo en el queue.

```
        ...
```

```
        SLEEP(...) // ?
```

```
        exit(0).
```

// padre →


```
close(fd1[READ]);  
close(fd2[WRITE]);
```

```
char* b2[1024];
```

```
char* res[1024];
```

```
while(1) {
```

```
    read(STDIN, &b2, sizeof(b2)); * fgets, scanf
```

```
    write(fd1[WRITE], &b2, sizeof(b2));
```

```
    read(fd2[READ], &res, sizeof(res));
```

```
    if (res == "YENDO")
```

```
        break; // strcmp
```

```
}
```

```
wait(NULL);
```

```
write(STDOUT, "LLEGAMOS", 9); * printf.
```

```
exit(0);
```

```
}
```